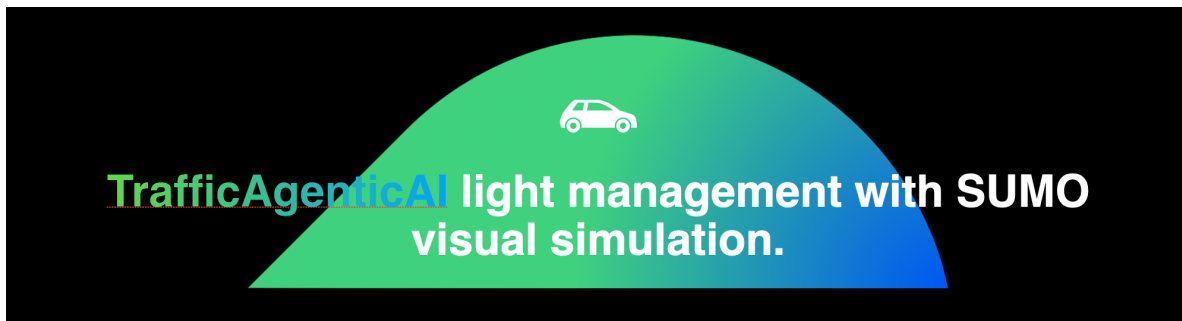




Raffaele Neri
raffaele.neri2@studio.unibo.it
Università di Bologna
Gravina in Puglia , Puglia, Italia

Matteo Melotti
matteo.melotti5@studio.unibo.it
Università di Bologna
Bologna, Emilia Romagna, Italia

Marco Crisafulli
marco.crisafulli@studio.unibo.it
Università di Bologna
Gela, Sicilia, Italia

1 Introduzione

Nel contesto urbano moderno, la gestione del **traffico** rappresenta una delle sfide più rilevanti per la qualità della vita, la sostenibilità ambientale e l'efficienza delle infrastrutture cittadine. I sistemi semaforici tradizionali, basati su **temporizzazioni fisse**, risultano spesso poco efficaci poiché non riescono ad adattarsi dinamicamente alle reali condizioni della rete stradale, causando congestione, aumento dei tempi di percorrenza, consumo energetico e spreco di risorse.

In questo elaborato si propone lo sviluppo di un **sistema intelligente multi-agente** per il controllo adattivo dei semafori urbani. L'architettura è progettata secondo un modello distribuito e gerarchico, in cui il controllo locale degli incroci viene combinato con un coordinamento globale dell'intera rete.

Al livello più basso si trova l'ambiente di simulazione **SUMO**, che riproduce il comportamento del traffico urbano e mantiene lo stato fisico della rete stradale. La comunicazione con SUMO avviene tramite **TraCI** (*Traffic Control Interface*), che consente di leggere in tempo reale informazioni come code, veicoli presenti sulle corsie, stati semaforici e lunghezze delle lane, oltre ad applicare comandi sui semafori.

Sopra il livello simulativo opera il **MCP Server**, che espone agli agenti strumenti operativi sotto forma di tool. Attraverso MCP, gli agenti possono calcolare lo **stress_index**, modificare la durata di una fase semaforica o cambiare la fase attiva di un semaforo. In questo modo il modello linguistico non interagisce direttamente con SUMO, ma utilizza strumenti controllati e deterministici.

Il livello successivo è composto dai **Traffic Agent**, ciascuno responsabile di una porzione della rete stradale. Ogni agente riceve una topologia locale dal backend, valuta lo stato del traffico della propria area, invoca i tool MCP necessari e produce una decisione tattica. Le decisioni locali possono includere il mantenimento della configurazione corrente, l'adattamento della durata semaforica o il cambio di fase nei casi di congestione elevata.

Al vertice della gerarchia è presente un **Orchestrator Service**, responsabile del coordinamento strategico tra gli agenti locali. L'orchestratore raccoglie gli output degli agenti tramite API FastAPI, salva e recupera lo storico degli stress index, valuta l'andamento della congestione e genera direttive globali per il ciclo successivo. Le direttive non agiscono direttamente sui semafori, ma guidano il comportamento degli agenti locali.

L'intera architettura è containerizzata e orchestrata tramite **Kubernetes/Minikube**, così da rendere il sistema scalabile, modulare e facilmente distribuibile. I servizi principali, come Agent Service,

Orchestrator Service, Backend, MCP Server e monitoraggio, comunicano tramite endpoint HTTP e sono gestiti come componenti indipendenti.

A supporto del sistema è presente un backend persistente basato su **FastAPI** e **SQLite**, utilizzato per salvare le topologie degli agenti e lo storico degli stress index. Inoltre, lo stack di monitoraggio basato su **Grafana**, **Prometheus**, **Loki** e **Infinity Query** consente di osservare metriche infrastrutturali, log applicativi e andamento temporale delle prestazioni degli agenti.

L'obiettivo del progetto è ridurre i tempi di attesa, migliorare la fluidità del traffico e dimostrare come l'integrazione tra simulazione, architetture multi-agente, tool MCP, modelli linguistici e orchestrazione cloud-native possa contribuire a una gestione più efficiente, adattiva e scalabile delle infrastrutture urbane.

2 Background e Stato dell'Arte

La gestione del traffico urbano è tradizionalmente affidata a sistemi semaforici basati su temporizzazioni statiche o, nei casi più avanzati, su logiche adattive locali. Strumenti di simulazione come **SUMO** (*Simulation of Urban MObility*) consentono di modellare e analizzare scenari complessi, permettendo lo studio di strategie di controllo del traffico in ambienti realistici. Tuttavia, molte delle soluzioni esistenti operano secondo un paradigma **locale e reattivo**: ogni incrocio prende decisioni sulla base del traffico immediatamente osservabile, senza una visione coordinata su scala più ampia.

L'interfaccia **TraCI** svolge un ruolo fondamentale in questo contesto, in quanto permette di collegare algoritmi esterni alla simulazione SUMO, abilitando un controllo fine e in tempo reale degli elementi della rete. Grazie a TraCI è possibile implementare sistemi di controllo avanzati, in cui agenti software monitorano continuamente lo stato del traffico e intervengono modificando le fasi semaforiche o altri parametri della simulazione.

Le tecniche tradizionali di ottimizzazione del traffico si concentrano su approcci come il controllo adattivo basato su sensori, modelli matematici o algoritmi di ottimizzazione. Sebbene efficaci in contesti limitati, tali metodi mostrano difficoltà nel gestire scenari complessi e dinamici, dove le interazioni tra incroci distanti possono influenzare significativamente le prestazioni globali della rete.

Negli ultimi anni, i sistemi *multi-agente* hanno guadagnato attenzione come approccio promettente per la gestione distribuita del traffico. In tali sistemi, ogni incrocio o gruppo di incroci è modellato come un agente autonomo in grado di prendere decisioni e coordinarsi con altri agenti. Tuttavia, molte implementazioni risultano limitate a forme di coordinamento locale, senza una vera struttura gerarchica o capacità di pianificazione strategica globale.

Parallelamente, l'evoluzione dei *Large Language Models* (LLM) ha dimostrato la capacità di questi modelli di analizzare contesti complessi, integrare informazioni eterogenee e supportare processi decisionali ad alto livello¹. Questo apre nuove possibilità per il loro utilizzo nella gestione del traffico, non a livello di singolo incrocio, ma come supporto al coordinamento tra più agenti.

In questo contesto, emerge la necessità di un'architettura che combini la robustezza di regole statiche locali con capacità di coordinamento e pianificazione su scala più ampia. Il presente lavoro si inserisce in questa direzione, proponendo un sistema gerarchico in cui i singoli semafori operano secondo logiche tradizionali, gruppi di incroci sono gestiti da agenti locali e un orchestratore globale fornisce direttive strategiche per ottimizzare il traffico sull'intera rete, sfruttando l'interfaccia TraCI per l'interazione continua con l'ambiente simulato.

3 Architettura del Sistema

L'architettura proposta è basata su un sistema **multi-agente gerarchico** per il controllo adattivo del traffico urbano. Il sistema è composto da più livelli: simulazione SUMO, MCP Server, Traffic Agent, Orchestrator Service, backend persistente e infrastruttura Kubernetes.

A differenza di una struttura monolitica, ogni componente ha una responsabilità specifica. SUMO mantiene lo stato fisico della simulazione, gli agenti prendono decisioni locali, l'orchestratore coordina le strategie globali e il backend conserva dati persistenti come topologie e storico degli **stress_index**.

¹Recenti studi evidenziano l'impiego degli LLM in contesti di pianificazione, coordinamento multi-agente e ottimizzazione di sistemi complessi, mostrando come tali modelli possano fornire linee guida strategiche e migliorare il coordinamento tra entità distribuite.

Il sistema interagisce con la simulazione tramite **TraCI** (*Traffic Control Interface*), che consente di leggere lo stato del traffico e applicare comandi ai semafori. L'accesso a queste funzionalità non avviene direttamente dagli LLM, ma tramite il **MCP Server**, che espone tool controllati e deterministici.

3.1 Livello SUMO e TraCI

Il livello più basso dell'architettura è rappresentato da **SUMO**, che simula la rete stradale e il comportamento dei veicoli. Durante ogni step, la simulazione aggiorna lo stato di corsie, code, veicoli e semafori.

L'interazione tecnica con SUMO avviene tramite **TraCI**, che permette di:

- leggere il numero di veicoli presenti sulle corsie;
- rilevare veicoli fermi e code;
- ottenere informazioni sui semafori controllati;
- modificare fasi e durate semaforiche.

Nel sistema implementato, TraCI viene utilizzato dal livello simulativo e dai tool MCP per trasformare le decisioni degli agenti in comandi effettivi sulla simulazione.

3.2 MCP Server

Il **MCP Server** rappresenta il ponte tra il reasoning degli agenti e le funzionalità operative del sistema. Gli LLM non modificano direttamente SUMO, ma invocano tool MCP che eseguono operazioni controllate.

Per i Traffic Agent, i principali tool MCP sono:

- `compute_stress_index`, per calcolare il livello di congestione dell'area controllata;
- `compute_phase_duration`, per stimare una durata semaforica adattiva;
- `set_traffic_light_duration`, per modificare la durata della fase corrente;
- `set_traffic_light`, per cambiare la fase semaforica.

Per l'orchestratore, invece, MCP espone tool legati alla memoria del sistema:

- `save_agent_stress`, per salvare lo stress index prodotto dagli agenti;
- `get_recent_stress`, per recuperare lo storico recente degli stress index.

In questo modo MCP separa il livello decisionale dal livello operativo, rendendo il sistema più sicuro e modulare.

3.3 Traffic Agent

I **Traffic Agent** rappresentano il livello decisionale locale dell'architettura. Ogni agente gestisce una specifica porzione della rete stradale, definita tramite una topologia recuperata dal backend.

A ogni ciclo operativo, l'agente riceve dall'orchestratore lo step corrente e l'eventuale direttiva globale. Successivamente valuta lo stato della propria area tramite i tool MCP, calcola lo `stress_index` e produce una risposta strutturata con reasoning sintetico e azioni applicate.

In base al livello di stress, l'agente può non intervenire, modificare la durata della fase semaforica oppure cambiare fase nei casi più critici. Per evitare comportamenti instabili, i cambi fase sono regolati da un meccanismo di cooldown.

3.4 Orchestrator Service

L'**Orchestrator Service** rappresenta il livello strategico del sistema. Riceve il trigger iniziale dalla simulazione, individua gli agenti attivi nel cluster Kubernetes e li attiva in parallelo.

Dopo il reasoning locale, gli agenti inviano all'orchestratore i propri risultati tramite FastAPI. L'orchestratore attende la risposta di tutti gli agenti attivi, aggrega gli output ricevuti, salva gli **stress_index**, recupera lo storico recente e valuta l'evoluzione della congestione.

Il risultato del reasoning globale è una direttiva per ogni agente, come **prioritize_flow**, **hold_or_balance** o **reduce_aggressiveness**. Queste direttive non agiscono direttamente sui semafori, ma guidano le decisioni locali degli agenti nel ciclo successivo.

3.5 Backend Persistente

Il modulo **backend_server.py** implementa il backend persistente del sistema. Questo componente non prende decisioni sul traffico, ma svolge il ruolo di memoria centrale condivisa tra agenti, orchestratore e sistema di monitoraggio.

Il backend è realizzato tramite **FastAPI** e **SQLite**. All'avvio inizializza il database locale **traffic_state.db**, creando due tabelle principali:

- **topologies**, utilizzata per salvare le topologie assegnate ai singoli agenti;
- **stress_levels**, utilizzata per salvare lo storico degli **stress_index** prodotti dagli agenti.

La tabella **topologies** associa ogni **agent_id** alla relativa topologia in formato JSON. Questa topologia descrive la porzione di rete stradale controllata dall'agente e viene caricata nel backend durante la fase di inizializzazione del sistema.

```
cursor.execute("""
    CREATE TABLE IF NOT EXISTS topologies (
        agent_id TEXT PRIMARY KEY,
        topology_data TEXT,
        updated_at TIMESTAMP
    )
""")

cursor.execute("""
    CREATE TABLE IF NOT EXISTS stress_levels (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        agent_id TEXT,
        stress_value REAL,
        prompt_text TEXT,
        timestamp TIMESTAMP
    )
""")
```

Il backend espone endpoint REST per la gestione delle topologie. L'endpoint **POST /api/topology/{agent_id}** consente di caricare un file JSON e salvarlo nel database, mentre **GET /api/topology/{agent_id}** permette a ogni Agent Service di recuperare la propria topologia al momento dell'avvio.

Inoltre, il backend espone l'endpoint **GET /api/stress**, pensato per rendere disponibile lo storico degli **stress_index** ad altri componenti del sistema, come Grafana o plugin di interrogazione REST.

Questo consente di visualizzare l'andamento temporale dello stress per agente e costruire dashboard di monitoraggio.

Oltre alle API REST, `backend_server.py` monta anche un server MCP tramite **FastMCP**. Il server MCP viene esposto sotto il path `/mcp` con trasporto SSE, mentre FastAPI continua a gestire le rotte REST sotto `/api`.

I principali tool MCP esposti dal backend sono:

- `write_topology`, per salvare o aggiornare una topologia tramite tool call;
- `save_agent_stress`, per salvare nel database lo `stress_index` prodotto da un agente;
- `get_recent_stress`, per recuperare gli ultimi valori di stress salvati.

Nel flusso operativo, gli agenti interagiscono con il backend principalmente tramite API REST per recuperare la propria topologia. L'orchestratore, invece, utilizza i tool MCP per salvare e recuperare lo storico degli `stress_index`. Questa distinzione separa chiaramente la comunicazione applicativa dal tool calling usato durante il reasoning LLM.

Il backend fornisce quindi due canali distinti:

- **REST API**, usate per bootstrapping, recupero topologie e monitoraggio;
- **MCP tools**, usati dall'orchestratore per gestire la memoria storica degli stress index.

Questa memoria temporale consente al sistema di non basarsi soltanto sullo stato corrente del traffico, ma anche sull'evoluzione storica della congestione. In questo modo l'orchestratore può valutare trend, peggioramenti improvvisi e miglioramenti progressivi nelle diverse aree della rete.

3.6 Infrastruttura Kubernetes

Tutti i servizi principali sono containerizzati ed eseguiti all'interno di **Kubernetes/Minikube**. Kubernetes permette di:

- eseguire più agenti in parallelo;
- scalare il numero di Traffic Agent;
- mantenere servizi raggiungibili tramite DNS interno;
- riavviare automaticamente i pod in caso di errore;
- isolare e gestire i diversi componenti dell'architettura.

Questa scelta rende il sistema modulare e scalabile: aumentando il numero di agenti, è possibile controllare porzioni più ampie della rete urbana senza modificare la logica complessiva dell'architettura.

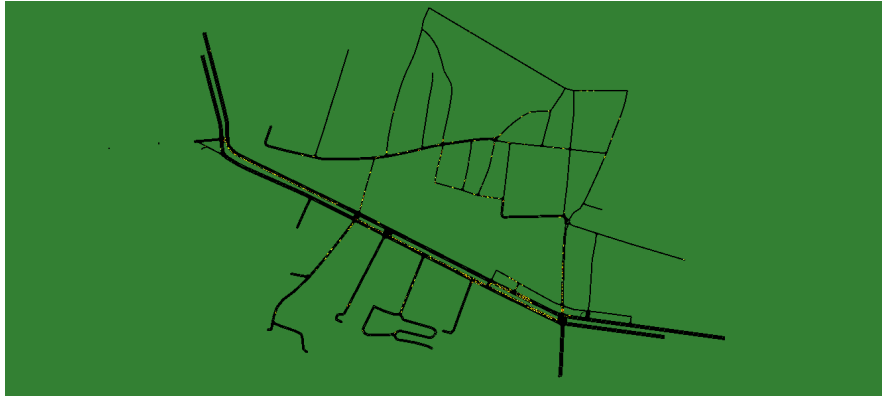
4 Pipeline di Costruzione delle Topologie

Per supportare l'architettura multi-agente è stata sviluppata una pipeline dedicata alla costruzione automatica delle topologie locali degli agenti. L'obiettivo è trasformare la rete stradale SUMO, contenuta nel file `.net.xml`, in un insieme di rappresentazioni compatte, ciascuna assegnata a un Traffic Agent.

La pipeline è composta da due moduli principali:

- `topology_builder.py`, che coordina l'intero processo di generazione;
- `topology_library.py`, che contiene la logica di estrazione, clustering e costruzione delle topologie.

A differenza di una precedente versione basata su TraCI, l'estrazione della rete avviene direttamente dal file `.net.xml`. Questa scelta permette di costruire le topologie senza avviare la simulazione, rendendo la fase di preprocessing più leggera, ripetibile e indipendente dal runtime SUMO.



4.1 Modulo `topology_builder.py`

Il modulo `topology_builder.py` rappresenta lo script principale importabile da altri componenti del sistema. La funzione centrale è `build_topologies()`, che esegue l'intera pipeline a partire dal nome della simulazione, dal numero di agenti desiderato e dalla cartella di output.

Il processo inizia individuando automaticamente il file `.net.xml` all'interno della cartella della simulazione. Successivamente viene richiamata la libreria di supporto per estrarre i dati della rete, calcolare i cluster e generare le topologie locali.

Le fasi principali sono:

1. ricerca del file `.net.xml` della simulazione;
2. estrazione di archi, semafori e incroci;
3. clustering spaziale della rete tramite K-Means;
4. costruzione delle topologie locali degli agenti;
5. salvataggio di un file JSON per ogni agente.

Uno snippet essenziale del modulo è il seguente:

```
def build_topologies(simulation_name: str, k: int, outdir: str):
    sim_folder = os.path.join(BASE_DIR, simulation_name)
    os.makedirs(outdir, exist_ok=True)

    net_file = find_net_file(sim_folder)

    edges_data, tls_data, junctions_data = tb.extract_network_data(net_file)
    clusters = tb.compute_clusters(k, tls_data, edges_data)
    topologies = tb.build_agent_topologies(clusters, junctions_data)

    for agent_id, topo in topologies.items():
        filepath = os.path.join(outdir, f"{agent_id}.json")
        with open(filepath, "w") as f:
            json.dump(topo, f, indent=2)

    return True
```

L'output finale è una cartella contenente file come `agent-0.json`, `agent-1.json`, ecc. Questi file vengono successivamente caricati nel backend e associati ai rispettivi Traffic Agent.

4.2 Modulo `topology_library.py`

Il modulo `topology_library.py` contiene la logica principale della pipeline. In particolare, implementa tre funzioni fondamentali:

- `extract_network_data()`;
- `compute_clusters()`;
- `build_agent_topologies()`.

La prima funzione, `extract_network_data()`, legge il file `.net.xml` in un'unica passata ed estrae tre insiemi di dati:

- `edges_data`, contenente gli archi stradali e le relative coordinate;
- `tls_data`, contenente gli incroci semaforizzati;
- `junctions_data`, contenente logiche semaforiche, fasi e connessioni tra archi.

Gli archi interni di SUMO, identificati da ID che iniziano con `:`, vengono ignorati. Per ogni arco valido viene calcolato un centroide medio a partire dalla geometria della corsia.

```
for edge in root.findall("edge"):
    e_id = edge.get("id")
    if not e_id or e_id.startswith(":"):
        continue

    lane = edge.find("lane")
    if lane is not None:
        shape_str = lane.get("shape", "")
        points = [tuple(map(float, p.split(","))) for p in shape_str.split()]
        x_center = sum(p[0] for p in points) / len(points)
        y_center = sum(p[1] for p in points) / len(points)
        edges_data.append({"id": e_id, "x": x_center, "y": y_center})
```

Successivamente vengono estratti gli incroci semaforizzati, le fasi semaforiche e le connessioni controllate, così da ricostruire la struttura logica degli incroci.

4.3 Clustering della Rete

La suddivisione della rete tra gli agenti avviene tramite l'algoritmo **K-Means**. Il clustering viene applicato alle coordinate spaziali di semafori e archi, in modo da ottenere aree geograficamente coerenti.

La funzione `compute_clusters()` costruisce un unico insieme di coordinate combinando semafori e archi, addestra K-Means con `k` cluster e assegna ogni elemento al cluster più vicino.

```
tls_coords = np.array([[t["x"], t["y"]] for t in tls_data])
edges_coords = np.array([[e["x"], e["y"]] for e in edges_data])
all_coords = np.vstack((tls_coords, edges_coords))

kmeans = KMeans(n_clusters=k, random_state=42, n_init="auto")
kmeans.fit(all_coords)
```

Il risultato è un dizionario in cui ogni chiave rappresenta un agente e contiene:

- la lista dei semafori assegnati;
- la lista degli archi appartenenti alla stessa area.

Questa fase permette di distribuire la rete tra più agenti, riducendo la complessità locale e mantenendo coerenza spaziale tra gli incroci controllati.

4.4 Costruzione delle Topologie Token-Slim

Dopo il clustering, la funzione `build_agent_topologies()` costruisce la topologia locale di ogni agente. L'obiettivo non è salvare l'intero grafo stradale in forma estesa, ma produrre una rappresentazione compatta e leggibile dall'LLM.

Ogni topologia contiene:

- `id`, identificativo dell'agente;
- `in`, archi di ingresso dall'esterno dell'area;
- `out`, archi di uscita verso l'esterno;
- `graph`, lista compatta delle connessioni controllate.

La rappresentazione delle connessioni segue il formato:

`E_IN > E_OUT(DEST)`

dove `E_IN` è l'arco in ingresso, `E_OUT` è l'arco in uscita e `DEST` indica l'incrocio raggiunto oppure `EXT` se la destinazione è esterna all'area dell'agente.

Esempio:

`J1: E1>E2(J2), E3>E4(EXT)`

Questa forma riduce significativamente il numero di token necessari per descrivere la rete nel prompt dell'agente, mantenendo comunque le informazioni essenziali per il reasoning locale.

4.5 Output della Pipeline

Il risultato finale della pipeline è un insieme di file JSON, uno per ogni agente. Ogni file descrive la porzione di rete controllata dall'agente in formato compatto.

Un esempio di struttura prodotta è:

```
{
  "id": "agent-0",
  "in": ["edge_in_1"],
  "out": ["edge_out_1"],
  "graph": [
    "junction_1: edge_a>edge_b(junction_2), edge_c>edge_d(EXT)"
  ]
}
```

Questi file vengono poi caricati nel backend tramite API REST e recuperati dagli Agent Service al momento dell'avvio. In questo modo ogni agente riceve soltanto la topologia della propria area, riducendo il contesto da fornire all'LLM e migliorando la scalabilità del sistema.

4.6 Vantaggi della Pipeline

La pipeline offre diversi vantaggi:

- non richiede l'avvio della simulazione per generare le topologie;
- suddivide automaticamente la rete tra più agenti;
- mantiene coerenza spaziale grazie al clustering K-Means;
- produce rappresentazioni compatte ottimizzate per gli LLM;

- rende il sistema scalabile aumentando semplicemente il parametro **k**.

Nel complesso, questa pipeline rappresenta il punto di collegamento tra la rete SUMO e l'architettura multi-agente, trasformando una mappa stradale complessa in contesti locali efficienti e utilizzabili dai Traffic Agent.

5 Sistema Multi-Agente e Orchestratore

Il sistema multi-agente è organizzato in due livelli principali: i *Traffic Agent*, responsabili del controllo locale degli incroci, e l'*Orchestrator Service*, responsabile del coordinamento globale del sistema.

Ogni agente opera su una specifica porzione della rete stradale e prende decisioni locali sulla base dello stato del traffico, dello stress index e delle direttive ricevute dall'orchestratore. L'orchestratore, invece, raccoglie gli output prodotti dagli agenti, analizza lo stato complessivo della rete e genera nuove direttive strategiche.

L'architettura è implementata attraverso moduli separati, in modo da distinguere chiaramente API service, logica decisionale e prompt policy.

5.1 Moduli del Traffic Agent

Ogni Traffic Agent è composto da tre moduli principali: `agent_service.py`, `agent_core.py` e `agent_policies.py`. Questa suddivisione separa il livello di servizio HTTP, il nucleo decisionale e le regole operative utilizzate dal modello linguistico.

Il modulo `agent_service.py` espone l'agente come servizio FastAPI. L'orchestratore comunica con l'agente tramite l'endpoint `/evaluate`, inviando lo step corrente della simulazione e l'eventuale direttiva globale. Per evitare che l'orchestratore rimanga bloccato durante il reasoning locale, l'elaborazione viene eseguita in background tramite `BackgroundTasks`.

```
@app.post("/evaluate")
async def evaluate_step(payload: EvaluatePayload, background_tasks: BackgroundTasks):
    background_tasks.add_task(
        app.state.agent_server.evaluate_and_push,
        step=payload.step,
        global_directive=payload.global_directive
    )

    return {
        "status": "processing",
        "message": "Reasoning avviato in background."
    }
```

Al termine del reasoning, l'agente prepara un payload contenente step, identificativo dell'agente e risultato prodotto. Questo risultato viene inviato all'orchestratore tramite l'endpoint `/stress_communication`. In questo modo l'agente non restituisce direttamente la decisione nella risposta HTTP iniziale, ma la comunica in modo asincrono quando il ciclo decisionale è terminato.

```

payload = {
    "step": step,
    "agent_id": self.agent_id,
    "result": result
}

resp = await self.http_client.post(
    f"{ORCHESTRATOR_URL}/stress_communication",
    json=payload
)

```

Durante l'inizializzazione, `agent_service.py` recupera anche la topologia assegnata all'agente dal backend. La topologia rappresenta la porzione di rete stradale controllata dal singolo agente e costituisce il contesto statico su cui viene costruito il prompt operativo.

Il modulo `agent_core.py` rappresenta il nucleo operativo del Traffic Agent. In esso vengono gestiti gli incroci assegnati, la connessione persistente con il MCP Server e il loop decisionale agentico. L'agente apre una connessione SSE verso il MCP Server e la riutilizza per tutta la durata del servizio, evitando di ricreare la connessione a ogni step.

All'interno di `agent_core.py` sono definiti i tool MCP disponibili per l'LLM. Il tool obbligatorio è `compute_stress_index`, utilizzato per calcolare il livello di congestione della zona. Gli altri tool permettono di calcolare una durata adattiva, modificare la durata della fase corrente o cambiare la fase semaforica.

```

REQUIRED_TOOLS = {"compute_stress_index"}

openai_tools = [
    "compute_stress_index",
    "compute_phase_duration",
    "set_traffic_light_duration",
    "set_traffic_light"
]

```

Il metodo `decide()` implementa il reasoning locale dell'agente. A ogni step viene costruito un prompt contenente identità dell'agente, incroci gestiti e direttiva globale ricevuta dall'orchestratore. La chat con l'LLM viene creata a ogni invocazione, così da evitare memoria conversazionale non necessaria e ridurre il consumo di token.

L'agente segue una logica tattica basata sullo stress index:

- in caso di stress basso, preferisce non intervenire;
- in caso di stress moderato, modifica preferibilmente la durata della fase corrente;
- in caso di stress alto, può cambiare fase semaforica se la congestione è persistente o in peggioramento.

Per evitare oscillazioni e cambi fase troppo ravvicinati, `agent_core.py` mantiene un meccanismo di cooldown locale. Ogni semaforo ha un tempo minimo tra due cambi fase successivi, definito da `min_phase_gap`.

```
self.last_phase_change = {}
self.min_phase_gap = 30
```

Se un cambio fase viene richiesto troppo presto rispetto all'ultimo intervento sullo stesso semaforo, l'azione viene saltata e viene restituito un risultato di tipo `skipped`. Questo migliora la stabilità del sistema e riduce comportamenti aggressivi.

Il modulo `agent_policies.py` contiene infine il prompt operativo dell'agente. In esso sono definite le soglie di stress, le regole tattiche, la mappatura tra policy e fasi semaforiche e il formato JSON finale dell'output. L'agente deve restituire una risposta strutturata contenente `stress_index`, reasoning sintetico e lista delle azioni eseguite.

Nel complesso, il Traffic Agent funziona come un servizio autonomo e scalabile: riceve uno step dall'orchestratore, valuta localmente la propria zona, invoca i tool MCP necessari e invia il risultato all'orchestratore per il coordinamento globale.

5.2 Moduli dell'Orchestratore

L'orchestratore è composto da tre moduli principali: `orchestrator_service.py`, `orchestrator_core.py` e `orchestrator_policies.py`. Questa suddivisione separa il livello di servizio HTTP, il nucleo decisionale globale e le regole strategiche utilizzate dal modello linguistico.

Il modulo `orchestrator_service.py` espone l'orchestratore come servizio FastAPI. La simulazione SUMO avvia il sistema tramite l'endpoint `/trigger_agentic`, passando lo step iniziale e l'identificativo della simulazione. Da questo punto parte il ciclo agentico continuo.

```
@app.post("/trigger_agentic")
async def trigger_agentic(payload: SumoTrigger):
    active_count = await app.state.orch.start_agentic_looping(
        payload.step,
        payload.simulation_id
    )

    return {
        "status": "success",
        "message": f"Agenti attivi svegliati: {active_count}."
    }
```

Durante l'inizializzazione, l'orchestratore utilizza le API Kubernetes per individuare automaticamente gli agenti attivi nel cluster. In questo modo il sistema non dipende da una lista statica di agenti, ma può adattarsi al numero di repliche disponibili.

```
Pods = self.k8s_api.list_namespaced_pod(
    NAMESPACE,
    label_selector=AGENT_LABEL
)
```

Una volta individuati gli agenti, il servizio li attiva in parallelo inviando a ciascuno lo step corrente e l'eventuale direttiva globale precedente. Ogni agente esegue il proprio reasoning locale e invia successivamente il risultato all'orchestratore tramite `/stress_communication`.

Il modulo `orchestrator_service.py` implementa anche una barriera di sincronizzazione. I risultati vengono raccolti in `step_results` e il workflow globale viene avviato solo quando tutti gli agenti attivi hanno completato il proprio ciclo.

```

self.step_results[step][agent_id] = result

received_agent_ids = set(self.step_results[step].keys())
missing_agents = self.expected_per_step[step] - received_agent_ids

if len(missing_agents) > 0:
    return False, list(missing_agents)

```

Quando l'ultimo agente ha inviato il risultato, l'orchestratore aggrega gli output locali e avvia il reasoning globale. Al termine viene calcolata una nuova direttiva globale e il sistema innesca automaticamente lo step successivo, mantenendo attivo il ciclo continuo.

```

next_step = step + 1
asyncio.create_task(self.trigger_all_agents(next_step))

```

Il modulo `orchestrator_core.py` contiene il nucleo decisionale dell'orchestratore. Riceve gli output aggregati degli agenti, li converte in un formato testuale compatto e costruisce il prompt strategico da inviare all'LLM. Questa compressione riduce il consumo di token e permette al modello di ragionare sullo stato globale della rete.

Durante il reasoning, l'orchestratore utilizza due tool MCP obbligatori:

```

REQUIRED_TOOLS = {
    "save_agent_stress",
    "get_recent_stress"
}

```

Il tool `save_agent_stress` salva lo stress index prodotto dagli agenti, mentre `get_recent_stress` recupera lo storico recente. Questo permette all'orchestratore di ragionare non solo sullo stato corrente, ma anche sull'evoluzione temporale della congestione.

Il metodo `decide()` genera una direttiva per ogni agente disponibile. L'orchestratore non applica direttamente modifiche ai semafori: produce invece direttive strategiche di alto livello, lasciando agli agenti locali la scelta dell'azione tattica più opportuna.

Le direttive prodotte possono essere:

- `prioritize_flow`, per dare priorità a una zona congestionata;
- `hold_or_balance`, per mantenere una gestione stabile e bilanciata;
- `reduce_aggressiveness`, per evitare cambi fase aggressivi in zone meno critiche.

Il modulo `orchestrator_policies.py` contiene il prompt strategico dell'orchestratore. In esso sono definite le soglie di stress, le regole di priorità, la gestione dei trend temporali, le regole anti-contraddizione e il formato JSON finale delle direttive.

Nel complesso, l'orchestratore agisce come livello strategico del sistema: coordina gli agenti, mantiene memoria dello storico recente, evita decisioni contraddittorie e garantisce continuità del ciclo operativo tra SUMO, Agent Service, MCP Server e controllo semaforico.

5.3 Flusso Operativo

Il flusso operativo del sistema segue una logica ciclica. SUMO innesca il sistema chiamando l'Orchestrator Service, che a sua volta attiva tutti gli Agent Service. Ogni agente esegue il proprio reasoning locale, comunica con il MCP Server per leggere o modificare lo stato della simulazione e invia il risultato all'orchestratore.

Quando tutti gli agenti hanno risposto, l'orchestratore aggrega gli output, valuta lo stato globale della rete e genera nuove direttive. Tali direttive vengono mantenute in memoria e inviate agli agenti nel ciclo successivo.

SUMO → Orchestrator Service → Agent Services → MCP Server → SUMO

5.4 Ruolo del MCP Server

Il **MCP Server** rappresenta il livello di interfaccia tra i componenti AI e la simulazione SUMO. Nel sistema proposto, gli agenti non accedono direttamente a TraCI o allo stato interno della simulazione, ma utilizzano tool MCP controllati. Questo consente di separare il reasoning LLM dalla logica operativa di accesso alla simulazione.

Il server viene implementato tramite **FastMCP** e inizializzato come gateway verso SUMO:

```
from fastmcp import FastMCP
from shared_memory import state

mcp = FastMCP("SUMO_Traffic_Gateway")
```

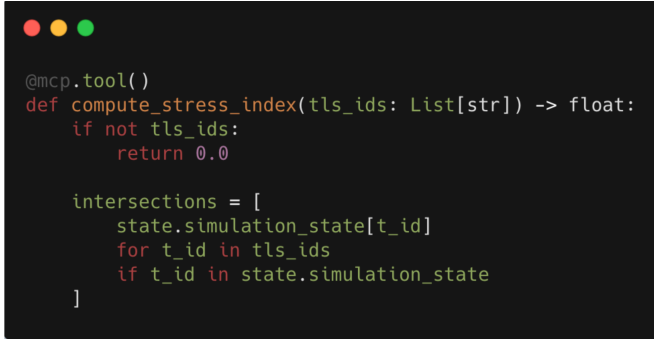
Il modulo importa una memoria condivisa, **state**, che contiene lo stato aggiornato della simulazione e la lista dei comandi pendenti da applicare ai semafori. In questo modo il MCP Server non modifica direttamente SUMO, ma legge dati già raccolti dalla simulazione e accoda comandi che verranno successivamente eseguiti dal livello TraCI.

La memoria condivisa contiene principalmente:

- **state.simulation_state**, con lo stato corrente degli incroci e delle corsie;
- **state.pending_commands**, con le azioni richieste dagli agenti e ancora da applicare.

5.4.1 Tool per il Calcolo dello Stress Index

Il tool principale utilizzato dagli agenti è **compute_stress_index**. Esso riceve la lista degli incroci controllati dall'agente e calcola un valore sintetico di congestione.



```
@mcp.tool()
def compute_stress_index(tls_ids: List[str]) -> float:
    if not tls_ids:
        return 0.0

    intersections = [
        state.simulation_state[t_id]
        for t_id in tls_ids
        if t_id in state.simulation_state
    ]
```

Il tool legge dalla memoria condivisa i dati aggiornati da SUMO. Per ogni incrocio considera numero totale di veicoli, veicoli in movimento, lunghezza delle corsie e coda complessiva.

La capacità teorica di un incrocio viene stimata sommando la capacità delle singole corsie. Ogni corsia viene approssimata assumendo una distanza media tra veicoli pari a $7.5m$:

$$Capacity_{lane} = \frac{Length_{lane}}{7.5}$$

Nel codice:

```
lane_length = l_data.get("length", 150)
capacita_totale_incrocio += (lane_length / 7.5)
```

Successivamente vengono calcolate due componenti: la saturazione della rete e il rapporto di veicoli fermi.

$$Saturation = \frac{TotalQueue}{TotalCapacity}$$

$$HaltingRatio = \frac{TotalVehicles - MovingVehicles}{TotalVehicles}$$

Lo stress dell'incrocio viene quindi calcolato come combinazione pesata:

$$Stress = 60 \cdot Saturation + 40 \cdot HaltingRatio$$

Nel codice:

```
saturation = min(inter.get("total_queue", 0) /  
                    capacita_totale_incrocio, 1.0)  
  
moving = sum(l["moving"] for l in inter.get("lanes_status", {}).values())  
halting_ratio = (total_v - moving) / total_v  
  
inter_stress = (saturation * 60) + (halting_ratio * 40)
```

Infine, lo `stress_index` restituito all'agente è la media degli stress calcolati sugli incroci della sua area:

```
final_stress = round(total_stress / len(intersections), 2)  
return final_stress
```

Questo valore diventa la metrica principale usata dal Traffic Agent per decidere se non intervenire, modificare la durata semaforica o cambiare fase.

5.4.2 Tool per il Controllo dei Semafori

Il MCP Server espone anche tool che permettono agli agenti di richiedere modifiche operative ai semafori.

Il primo tool è `set_traffic_light`, che accoda una richiesta di cambio fase:

```
@mcp.tool()  
def set_traffic_light(tl_id: str, phase_index: int):  
    state.pending_commands.append({  
        "tls_id": tl_id,  
        "phase_index": phase_index  
    })  
  
    return {  
        "status": "queued",  
        "tl_id": tl_id,  
        "phase_index": phase_index  
    }
```

Il secondo tool è `set_traffic_light_duration`, utilizzato per modificare la durata della fase corrente:

```
@mcp.tool()  
def set_traffic_light_duration(tl_id: str, duration: float):  
    state.pending_commands.append({  
        "type": "set_duration",  
        "tls_id": tl_id,  
        "duration": duration  
    })  
  
    return {"status": "queued"}
```

```

def run_mcp_server():
    try:
        mcp.run(
            transport='sse',
            host="0.0.0.0",
            port=8001
        )
    except Exception as e:
        print(f"Errore critico server: {e}")

```

Entrambi i tool non applicano immediatamente la modifica a SUMO, ma inseriscono il comando in `state.pending_commands`. Questo approccio rende il sistema più controllabile, perché l'applicazione effettiva dei comandi resta gestita dal componente che comunica con TraCI.

5.4.3 Tool per la Durata Adattiva

Quando lo stress è moderato, l'agente può preferire un intervento più morbido rispetto al cambio fase. Per questo viene usato il tool `compute_phase_duration`, che calcola una durata adattiva a partire dallo `stress_index`.

```

@mcp.tool()
async def compute_phase_duration(stress_index: float) -> float:
    duration = min(60, max(15, 20 + stress_index))
    return round(duration, 1)

```

La formula impone un limite minimo di 15 secondi e un massimo di 60 secondi, evitando durate troppo brevi o troppo lunghe. Il valore base è 20 secondi, aumentato in funzione dello stress rilevato.

$$Duration = \min(60, \max(15, 20 + StressIndex))$$

5.4.4 Avvio del Server MCP

Il server MCP viene avviato in modalità SSE sulla porta 8001. Questo consente agli agenti di mantenere una connessione persistente con il server e invocare i tool durante il ciclo decisionale.

L'utilizzo del trasporto SSE consente una comunicazione stabile tra Traffic Agent e MCP Server, evitando di ricreare la connessione a ogni chiamata tool.

Nel complesso, il MCP Server svolge quindi tre funzioni fondamentali: calcolare metriche di traffico, accodare comandi semaforici e fornire agli LLM un'interfaccia sicura e standardizzata verso SUMO.

5.5 Vantaggi Architettureali

La suddivisione in service, core e policies rende il sistema più leggibile e manutenibile. I service gestiscono la comunicazione HTTP e il ciclo di vita dei componenti, i core contengono la logica decisionale, mentre i file policies definiscono il comportamento LLM tramite prompt.

Questa organizzazione permette di modificare le strategie decisionali senza alterare l'infrastruttura applicativa, sostituire il provider LLM tramite `llm.connector.py` e scalare gli agenti in Kubernetes aumentando il numero di repliche.

6 Manager di Avvio dell'Infrastruttura

Il modulo `agenticTrafficManager.py` rappresenta lo script principale di avvio dell'intera infrastruttura multi-agente. Il suo ruolo è automatizzare le operazioni necessarie per preparare il sistema prima dell'esecuzione della simulazione agentica: generazione delle topologie, caricamento nel backend, avvio

del cluster Kubernetes, installazione del monitoraggio, build delle immagini container, scaling degli agenti e apertura dei port-forward locali.

Il manager viene eseguito da linea di comando specificando il nome della simulazione e il numero di agenti desiderato:

```
python3 trafficAgentic/agenticTrafficManager.py --simulation_name 2cross --k 2
```

Il parametro `k` è centrale, poiché viene utilizzato sia per suddividere la rete in `k` topologie locali sia per scalare lo `StatefulSet` degli agenti Kubernetes allo stesso numero di repliche.

6.1 Flusso Principale

La funzione principale dello script è `run_application()`, che coordina in sequenza le fasi di setup dell'infrastruttura.

```
def run_application(simulation_name, k):

    build_topologies(
        simulation_name=simulation_name,
        k=k,
        outdir=TOPOLOGY_DIR
    )

    for filename in os.listdir(TOPOLOGY_DIR):
        if filename.endswith(".json"):
            agent_id = "traffic-" + os.path.splitext(filename)[0]

            with open(os.path.join(TOPOLOGY_DIR, filename), "rb") as f:
                files = {"file": (filename, f, "application/json")}
                requests.post(f"{BACKEND_URL}/{agent_id}", files=files)

    setup_minikube()
    setup_monitoring()

    images_rebuilt = build_images()
    apply_k8s(images_rebuilt, k)

    start_port_forward("orchestrator-service", 8080, 8080, "default")
    start_port_forward("monitoring-stack-grafana", 3000, 80, "monitoring")
```

Il flusso mostra come il manager colleghi tra loro le principali componenti del sistema: prima genera le topologie locali, poi le carica nel backend tramite API REST, successivamente prepara l'infrastruttura Kubernetes e infine espone localmente i servizi principali.

6.2 Generazione e Caricamento delle Topologie

La prima fase riguarda la costruzione delle topologie degli agenti. La funzione `build_topologies()` suddivide la rete stradale in base al numero di agenti richiesto e produce un file JSON per ciascun agente.

Dopo la generazione, ogni file viene caricato nel backend persistente tramite una richiesta HTTP verso l'endpoint `/api/topology/{agent_id}`. In questo modo, quando i pod degli agenti verranno avviati, potranno recuperare automaticamente la propria topologia dal backend.

```

agent_id = "traffic-" + os.path.splitext(filename)[0]

with open(file_path, "rb") as f:
    files = {"file": (filename, f, "application/json")}
    response = requests.post(
        f"{BACKEND_URL}/{agent_id}",
        files=files
    )

```

Questa fase collega la pipeline topologica al sistema distribuito: le topologie non vengono inserite manualmente nei pod, ma vengono salvate centralmente e recuperate dagli agenti al momento dell'avvio.

6.3 Preparazione del Cluster Kubernetes

Il manager verifica che il cluster Minikube sia attivo e, se necessario, lo avvia automaticamente. Successivamente installa lo stack di monitoraggio tramite Helm e prepara le immagini container dell'orchestratore e degli agenti.

L'applicazione dei manifest Kubernetes avviene tramite il comando `kubectl apply`, che crea o aggiorna le risorse definite nella cartella di configurazione.

```

subprocess.run(
    ["kubectl", "apply", "-f", K8S_DIR],
    check=True,
    stdout=subprocess.DEVNULL
)

```

Questa fase permette di creare i workload principali del sistema, tra cui orchestratore, agenti, servizi interni, configurazioni e risorse necessarie all'esecuzione nel cluster.

6.4 Scaling degli Agenti

Una volta applicati i manifest, il manager scala imperativamente lo `StatefulSet` degli agenti al numero di repliche richiesto dal parametro `k`.

```

subprocess.run(
    ["kubectl", "scale", "statefulset/traffic-agent",
     f"--replicas={num_replicas}"],
    check=True,
    stdout=subprocess.DEVNULL
)

```

Questa scelta consente di modificare rapidamente il numero di agenti attivi senza cambiare manualmente i manifest YAML. Aumentando `k`, il sistema genera più topologie e avvia più pod agente, mantenendo coerenza tra partizionamento della rete e deployment Kubernetes.

6.5 Monitoraggio e Port-Forwarding

Il manager installa lo stack di monitoraggio, composto da Prometheus, Grafana, Loki e Promtail. Grafana viene esposta localmente tramite port-forward sulla porta 3000, mentre l'orchestratore viene esposto sulla porta 8080.

```

start_port_forward("orchestrator-service", 8080, 8080, "default")
start_port_forward("monitoring-stack-grafana", 3000, 80, "monitoring")

```

Il port-forward consente di accedere ai servizi interni del cluster direttamente dalla macchina locale. In particolare, l'orchestratore può essere raggiunto dalla simulazione SUMO, mentre Grafana può essere aperta dal browser per osservare metriche, log e andamento dello `stress_index`.

6.6 Ruolo del Manager nel Sistema

Il manager non partecipa direttamente al reasoning agentico e non prende decisioni sul traffico. Il suo compito è preparare l'ambiente in modo ripetibile e automatizzato.

In sintesi, `agenticTrafficManager.py` esegue le seguenti operazioni:

- genera le topologie locali degli agenti;
- carica le topologie nel backend tramite API REST;
- avvia o verifica il cluster Minikube;
- installa lo stack di monitoraggio;
- costruisce le immagini container;
- applica i manifest Kubernetes;
- scala il numero di agenti in base al parametro `k`;
- espone localmente orchestratore e Grafana.

Questa automazione rende l'infrastruttura più semplice da avviare, testare e scalare, riducendo il numero di operazioni manuali necessarie per eseguire l'intero sistema multi-agente.

7 Comunicazione e Integrazione con SUMO

L'integrazione con SUMO è realizzata tramite una struttura modulare che separa il motore di simulazione dalla logica decisionale degli agenti. SUMO mantiene lo stato fisico della rete stradale, mentre TraCI consente di leggere i dati della simulazione e applicare comandi sui semafori in tempo reale.

Nel sistema sviluppato, gli agenti non interagiscono direttamente con TraCI. L'accesso alla simulazione è mediato dal `simulationManager.py` e dal **MCP Server**. Il primo comunica realmente con SUMO tramite TraCI, mentre il secondo espone agli agenti tool controllati per leggere lo stato della simulazione e richiedere azioni operative.

7.1 Ruolo di SUMO e TraCI

SUMO rappresenta il motore della simulazione. A ogni step aggiorna lo stato di veicoli, corsie, code e semafori. TraCI (*Traffic Control Interface*) fornisce l'interfaccia tecnica utilizzata dal Simulation Manager per interrogare e modificare tale stato.

Attraverso TraCI è possibile:

- leggere il numero di veicoli presenti su una corsia;
- rilevare veicoli fermi e code;
- ottenere informazioni su semafori e fasi attive;
- modificare durata e fase dei semafori;
- avanzare la simulazione step-by-step.

Nel progetto, TraCI viene utilizzato direttamente solo dal `simulationManager.py`. Gli agenti e l'LLM non accedono a TraCI, ma operano tramite tool MCP e memoria condivisa.

7.2 Simulation Manager

Il modulo `simulationManager.py` rappresenta il componente che avvia e controlla la simulazione SUMO. Esso costituisce il ponte operativo tra simulazione, MCP Server e orchestratore globale.

All'avvio, il Simulation Manager avvia il server MCP in un thread separato e inizializza la simulazione SUMO tramite TraCI.

```
api_thread = threading.Thread(target=run_mcp_server, daemon=True)
api_thread.start()

traci.start(sumo_cmd)
initialize_static_data()
```

La funzione `initialize_static_data()` raccoglie informazioni statiche della rete, come la lunghezza delle corsie controllate dai semafori. Questi dati vengono salvati nella memoria condivisa e utilizzati successivamente dal tool MCP `compute_stress_index` per stimare la capacità degli incroci.

Durante l'esecuzione, a ogni step della simulazione, il manager legge tramite TraCI lo stato aggiornato dei semafori e delle corsie. Per ogni incrocio semaforizzato vengono calcolati il numero totale di veicoli, la coda complessiva e lo stato delle corsie.

```
state.simulation_state[tls_id] = {
    "id": tls_id,
    "total_vehicles": tot_vehicles,
    "total_queue": tot_queue,
    "lanes_status": lanes_status
}
```

Queste informazioni vengono salvate in `state.simulation_state`, cioè la memoria condivisa letta dal MCP Server. In questo modo gli agenti non interrogano direttamente SUMO, ma richiedono al MCP Server dati già aggiornati e strutturati.

Il Simulation Manager si occupa anche di applicare alla simulazione i comandi richiesti dagli agenti. Tali comandi vengono prima inseriti dal MCP Server in `state.pending_commands`; successivamente il manager li legge e li traduce in chiamate TraCI.

```
while state.pending_commands:
    cmd = state.pending_commands.pop(0)

    if cmd.get("type") == "set_duration":
        traci.trafficlight.setPhaseDuration(
            tl_id,
            cmd["duration"]
        )
    else:
        traci.trafficlight.setPhase(
            tl_id,
            target_phase
        )
```

Prima di applicare un cambio fase, il manager verifica che la fase richiesta sia valida rispetto al numero di fasi disponibili per quel semaforo. Inoltre introduce una transizione sicura: se la fase corrente

contiene un verde attivo, il sistema evita il passaggio brusco alla nuova fase e forza prima la chiusura della fase corrente. Questo riduce comportamenti instabili e frenate improvvise nella simulazione.

Infine, dopo un intervallo iniziale configurabile tramite `decision_interval`, il Simulation Manager attiva il ciclo agentico contattando l'orchestratore tramite FastAPI.

```
requests.post(
    ORCHESTRATOR_URL,
    json={
        "step": 0,
        "simulation_id": simulation_name
    },
    timeout=10.0
)
```

Da questo momento l'orchestratore avvia gli agenti, raccoglie i risultati e continua il ciclo decisionale. Il Simulation Manager rimane responsabile dell'avanzamento della simulazione, dell'aggiornamento della memoria condivisa e dell'applicazione dei comandi prodotti dal sistema multi-agente.

Il flusso di lettura dello stato può essere riassunto come segue:

SUMO → TraCI → Simulation Manager → `state.simulation_state` → MCP Server → Traffic Agent

Il flusso di applicazione delle azioni è invece:

Traffic Agent → MCP Server → `state.pending_commands` → Simulation Manager → TraCI → SUMO

7.3 Astrazione tramite MCP

Il **MCP Server** espone agli agenti tool standardizzati per leggere lo stato della simulazione e richiedere azioni operative. Tuttavia, nel codice attuale, MCP non chiama direttamente TraCI: legge dalla memoria condivisa aggiornata dal Simulation Manager e scrive comandi nella coda `state.pending_commands`.

Per i Traffic Agent, i tool principali sono:

- `compute_stress_index`, per calcolare il livello di congestione della zona gestita leggendo `state.simulation_state`;
- `compute_phase_duration`, per stimare una durata semaforica adattiva;
- `set_traffic_light_duration`, per accodare una modifica della durata della fase corrente;
- `set_traffic_light`, per accodare un cambio fase semaforica.

Il flusso operativo corretto non è quindi diretto verso TraCI, ma passa dalla memoria condivisa:

Traffic Agent → MCP Tool → Shared Memory → Simulation Manager → TraCI → SUMO

Nel caso del calcolo dello stress, il MCP Server legge `state.simulation_state` e restituisce il valore all'agente. Nel caso delle azioni sui semafori, il MCP Server scrive un comando in `state.pending_commands`; sarà poi il Simulation Manager ad applicarlo realmente su SUMO tramite TraCI.

Il risultato del reasoning locale viene infine inviato dal Traffic Agent all'orchestratore tramite FastAPI, in particolare attraverso l'endpoint `/stress_comunicazione`.

7.4 Ruolo della Comunicazione nel Sistema

L'utilizzo combinato di SUMO, TraCI, MCP, memoria condivisa e FastAPI consente di strutturare il flusso informativo in modo chiaro. SUMO fornisce lo stato simulato della rete, TraCI consente l'accesso tecnico alla simulazione, il Simulation Manager aggiorna la memoria condivisa e applica i comandi, MCP espone tool controllati agli agenti, mentre FastAPI permette ai servizi distribuiti di comunicare tra loro.

Questa architettura migliora modularità e sicurezza del sistema: gli LLM non modificano direttamente la simulazione, ma operano tramite strumenti definiti; gli agenti mantengono autonomia locale; l'orchestratore riceve risultati aggregati e produce direttive strategiche per il ciclo successivo.

8 Monitoraggio del Sistema

Per monitorare il comportamento dell'infrastruttura viene utilizzato **Grafana**, una piattaforma open-source per la visualizzazione e l'analisi di metriche in tempo reale. L'obiettivo è osservare sia lo stato del cluster Kubernetes/Minikube sia le metriche applicative prodotte dagli agenti, in particolare lo `stress_index`.

Grafana consente di costruire dashboard interattive collegandosi a diverse sorgenti dati. Nel sistema proposto viene integrato con **Prometheus**, per la raccolta delle metriche infrastrutturali e HTTP, con **Loki**, per l'analisi dei log applicativi, e con **Infinity Query**, per interrogare endpoint REST e visualizzare dati provenienti dal backend applicativo.

L'installazione dello stack di monitoraggio avviene tramite script Python e **Helm**, il package manager di Kubernetes. Questa scelta permette di automatizzare il deployment e di rendere il monitoraggio parte integrante dell'architettura containerizzata.

8.1 Installazione dello Stack Prometheus e Grafana

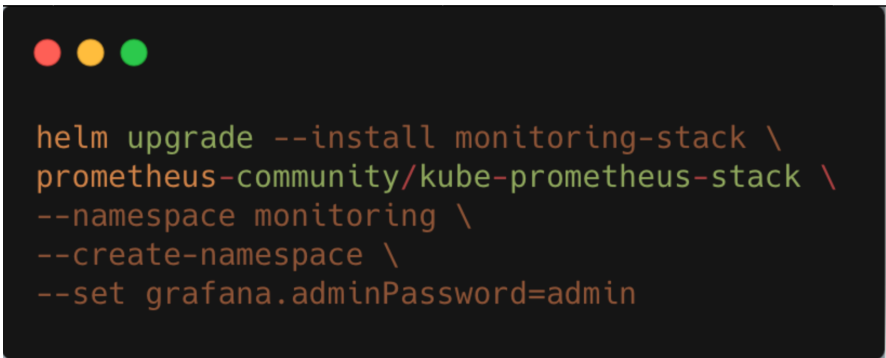
L'installazione dello stack di monitoraggio viene automatizzata tramite la funzione `setup_monitoring()`, integrata nello script principale di gestione dell'infrastruttura.

La funzione utilizza Helm per installare o aggiornare automaticamente lo stack **Prometheus + Grafana** all'interno del cluster Minikube.

Il processo avviene in tre fasi:

1. aggiunta del repository Helm ufficiale `prometheus-community`;
2. aggiornamento degli indici dei chart disponibili;
3. installazione o aggiornamento del chart `kube-prometheus-stack` nel namespace `monitoring`.

Il comando principale utilizzato è:



```
helm upgrade --install monitoring-stack \
prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--create-namespace \
--set grafana.adminPassword=admin
```

L'opzione `upgrade --install` consente di usare lo stesso comando sia per una prima installazione sia per aggiornare una configurazione già esistente. Questo rende il deployment ripetibile e idempotente.

Il namespace `monitoring` viene creato automaticamente se non esiste già, separando le risorse di osservabilità dal resto dell'applicazione.

La password `admin` viene impostata solo per ambiente di sviluppo e test; in un deployment reale dovrebbe essere sostituita con un Secret Kubernetes o con una configurazione più sicura.

Nel caso in cui Helm non sia installato o si verifichi un errore durante il deployment, la funzione non interrompe l'intera applicazione, ma produce un warning. In questo modo il sistema multi-agent può comunque essere avviato anche se il monitoraggio non è disponibile.

8.2 Connettività di Grafana con il Sistema

Grafana si collega al sistema attraverso tre canali principali.

Il primo canale riguarda le metriche infrastrutturali. I servizi FastAPI degli agenti e dell'orchestratore espongono endpoint monitorabili, mentre Prometheus effettua lo scraping periodico degli endpoint e raccoglie informazioni su stato dei servizi, richieste HTTP, tempi di risposta e disponibilità dei componenti.

Il secondo canale riguarda i log applicativi. Loki raccoglie i log prodotti dai container Kubernetes e li rende interrogabili da Grafana. Questo permette di osservare errori, tool call MCP, risposte degli agenti, fallback dell'orchestratore e problemi di comunicazione tra servizi.

Il terzo canale riguarda le metriche applicative salvate nel backend. Gli agenti producono uno `stress_index` a ogni ciclo decisionale; tale valore viene inviato all'orchestratore e salvato nel database SQLite tramite i tool MCP. Grafana può recuperare questi dati tramite endpoint REST, ad esempio usando Infinity Query.

Il flusso logico per le metriche applicative è:

Agenti → Orchestratore → Backend SQLite → API REST → Grafana

Per le metriche infrastrutturali:

Pod Kubernetes → Endpoint HTTP → Prometheus → Grafana

Per i log:

Pod Kubernetes → Loki → Grafana

8.3 Manifesto e Deployment di Grafana

Il deployment di Grafana viene gestito tramite Helm e manifesti Kubernetes. Helm consente di installare Grafana nel cluster Minikube in modo ripetibile, configurando automaticamente pod, service, datasource e plugin necessari.

Il manifesto di Grafana definisce gli elementi fondamentali per il funzionamento del servizio:

- il pod Grafana, che esegue l'interfaccia web e il motore delle dashboard;
- il service Kubernetes, che espone Grafana all'interno del cluster;
- le configurazioni dei datasource, come Prometheus, Loki e API REST;
- eventuali plugin aggiuntivi per interrogare sorgenti esterne;
- le credenziali e le impostazioni di accesso alla dashboard.

Nel nostro caso, Grafana viene utilizzato come punto centrale di osservabilità. Il suo manifesto permette di mantenere la configurazione versionata e replicabile, così che l'ambiente di monitoraggio possa essere ricreato automaticamente insieme al resto dell'infrastruttura.

Questa scelta è coerente con l'approccio containerizzato del progetto: anche il monitoraggio viene eseguito dentro Kubernetes, beneficia dei meccanismi di gestione dei pod e può essere riavviato o riconfigurato senza interventi manuali complessi.

8.4 Metriche Monitorate

Le principali metriche monitorate sono:

- stato e disponibilità dei pod Kubernetes;
- tempi di risposta degli endpoint FastAPI;
- numero di richieste ricevute da agenti e orchestratore;
- andamento dello `stress_index` per agente;
- storico temporale delle condizioni di traffico;
- errori applicativi e log dei container;
- eventuali anomalie o picchi improvvisi di congestione.

Grazie a queste dashboard, il sistema può essere valutato non solo in termini funzionali, ma anche dal punto di vista operativo. Il monitoraggio consente infatti di individuare colli di bottiglia, verificare il corretto comportamento degli agenti e analizzare l'efficacia delle strategie prodotte dall'orchestratore.

8.5 Loki e Infinity Query

Oltre a Prometheus, lo stack di monitoraggio viene esteso con datasource aggiuntivi come **Loki** e **Infinity Query**, utili per osservare sia i log applicativi sia metriche esposte tramite API REST.

Loki è un sistema di raccolta e interrogazione dei log progettato per integrarsi direttamente con Grafana. A differenza di Prometheus, che lavora principalmente su metriche numeriche, Loki permette di analizzare i log prodotti dai container Kubernetes. Nel sistema può essere utilizzato per osservare i log generati da:

- Traffic Agent;
- Orchestrator Service;
- Backend Server;
- MCP Server;
- pod Kubernetes in esecuzione.

Questo consente di verificare errori, tool call MCP, risposte degli agenti, fallback dell'orchestratore e problemi di comunicazione tra servizi. Loki è quindi particolarmente utile per il debugging e per ricostruire il comportamento del sistema durante una simulazione.

8.5.1 Configurazione di Loki e Infinity Query come Datasource Grafana

Per integrare log applicativi e metriche REST all'interno delle dashboard, Grafana viene configurato con due datasource aggiuntivi: **Loki** e **Infinity Query**. La configurazione viene definita nel file dei valori Helm, così che i collegamenti vengano creati automaticamente durante il deployment.

```

grafana:
  dnsPolicy: "None"
  dnsConfig:
    nameservers:
      - 10.96.0.10
      - 8.8.8.8
    searches:
      - monitoring.svc.cluster.local
      - svc.cluster.local
      - cluster.local
    options:
      - name: ndots
        value: "2"

  plugins:
    - yesoreyeram-infinity-datasource

  additionalDataSources:
    - name: loki
      type: loki
      access: proxy
      url: http://loki:3100
      jsonData:
        httpHeaderName1: 'X-Scope-OrgID'
      secureJsonData:
        httpHeaderValue1: 'local'

    - name: Infinity-API
      type: yesoreyeram-infinity-datasource
      access: proxy
      isDefault: false
      jsonData:
        allowedHosts:
          - http://host.minikube.internal:8000

```

La configurazione DNS permette a Grafana di risolvere correttamente sia i servizi interni al cluster Kubernetes sia il backend locale raggiungibile tramite `host.minikube.internal`. Il plugin `yesoreyeram-infinity-datasource` abilita invece l'interrogazione di endpoint REST, JSON o CSV direttamente dalle dashboard.

Il datasource **Loki** punta al servizio interno `http://loki:3100`. L'opzione `access: proxy` indica che Grafana inoltra le richieste al datasource tramite il proprio backend. L'intestazione `X-Scope-OrgID` viene utilizzata perché Loki è configurato con autenticazione multi-tenant abilitata; nel progetto il tenant utilizzato è `local`.

```

loki:
  auth_enabled: true

```

Il datasource **Infinity-API** viene invece utilizzato per collegare Grafana agli endpoint REST del backend applicativo. In particolare, permette di recuperare dati salvati nel database SQLite, come lo storico degli `stress_index`, senza passare da Prometheus.

```

allowedHosts:
  - http://host.minikube.internal:8000

```

Questa configurazione è necessaria perché Grafana viene eseguito dentro Minikube, mentre il backend può essere esposto sulla macchina host. In questo modo le dashboard possono interrogare endpoint come `/api/stress` e visualizzare l'andamento temporale dello stress per ogni agente.

Per semplicità e compatibilità con l'ambiente Minikube, Loki viene eseguito in modalità `SingleBinary`. In questa modalità le componenti di lettura, scrittura e backend non vengono scalate separatamente, ma sono concentrate in un'unica istanza.

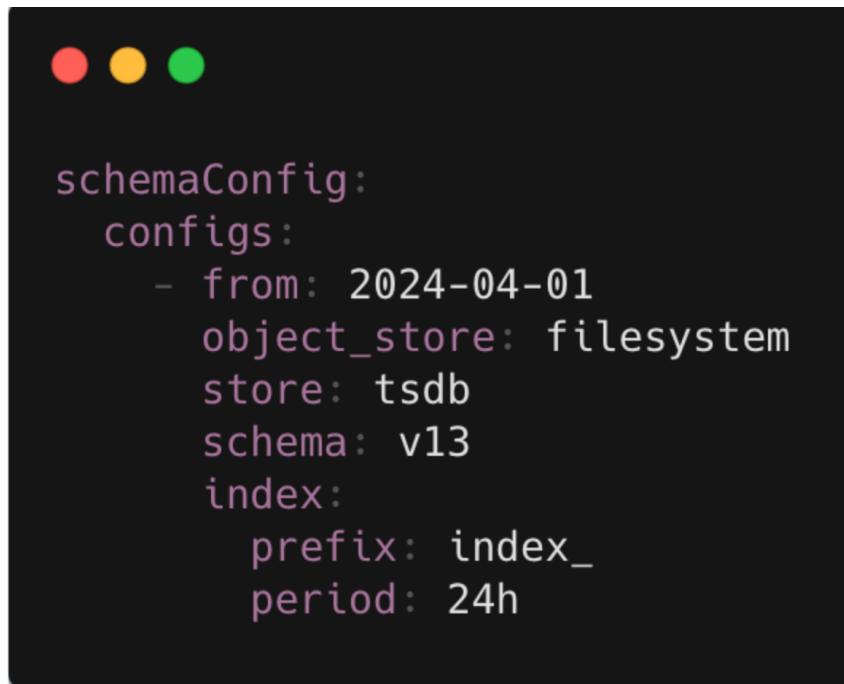
```
deploymentMode: SingleBinary

write:
  replicas: 0
read:
  replicas: 0
backend:
  replicas: 0
singleBinary:
  replicas: 1
```

La persistenza dei log viene gestita tramite filesystem locale:

```
loki:
  storage:
    type: 'filesystem'
    bucketNames:
      chunks: chunks
      ruler: ruler
      admin: admin
```

Lo schema di indicizzazione utilizza TSDB con schema `v13`, con indici giornalieri:



```
schemaConfig:
  configs:
    - from: 2024-04-01
      object_store: filesystem
      store: tsdb
      schema: v13
      index:
        prefix: index_
        period: 24h
```

Nel complesso, **Prometheus**, **Loki** e **Infinity Query** coprono tre esigenze diverse: Prometheus raccoglie metriche numeriche dal cluster e dai servizi, Loki raccoglie e interroga i log dei container, mentre Infinity Query consente a Grafana di leggere dati applicativi esposti dal backend tramite API REST.

9 Risultati Sperimentali

Per valutare il comportamento del sistema multi-agent è stata eseguita una simulazione distribuita del traffico urbano utilizzando SUMO, Traffic Agent specializzati e un orchestratore globale basato su LLM.

L'obiettivo dell'esperimento è stato osservare:

- la stabilità dell'infrastruttura Kubernetes;
- il comportamento distribuito degli agenti;
- l'andamento dello stress index;
- la scalabilità del sistema al crescere del numero di agenti.

9.1 Health Level e Stabilità Operativa

La dashboard Grafana dedicata allo stato operativo mostra il numero di restart e l'uptime dei Traffic Agent e dell'orchestratore.

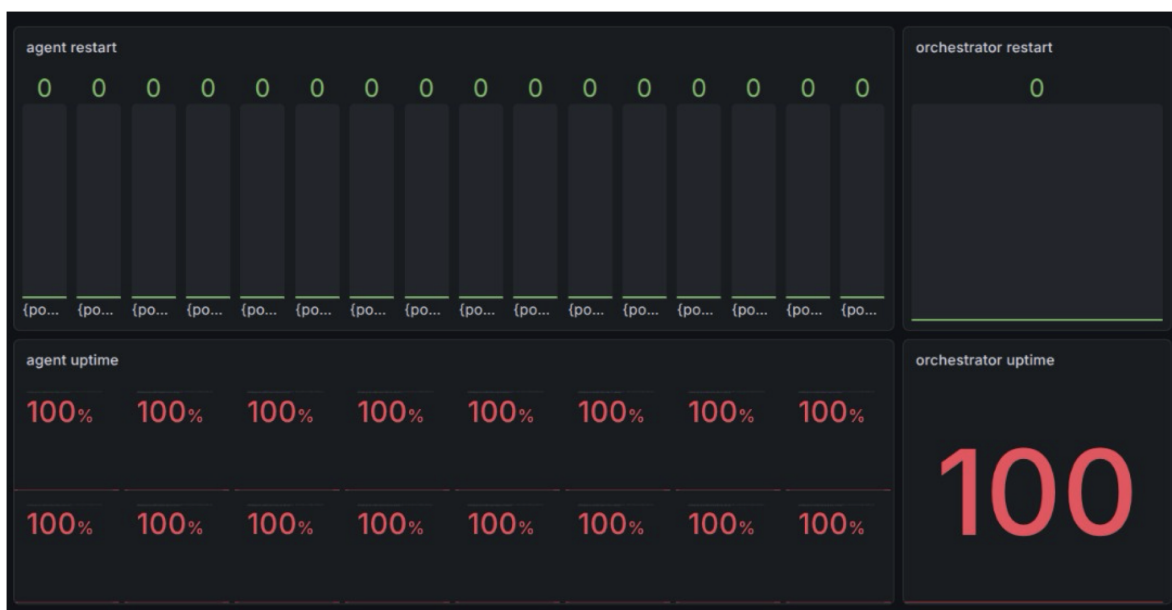


Figure 1: Dashboard Grafana per il monitoraggio dello stato operativo

L'health level è stato costruito considerando:

- uptime dei pod Kubernetes;
- assenza di restart;
- disponibilità dei servizi;
- stato operativo dell'orchestratore.

Durante la simulazione il sistema ha mantenuto uptime pari al 100% senza restart significativi, mostrando buona stabilità operativa.

9.2 Stress Index Distribuito

Ogni Traffic Agent calcola periodicamente uno **stress_index** sulla base della congestione locale osservata nella propria topologia.

Le metriche vengono raccolte tramite Prometheus e visualizzate in Grafana.

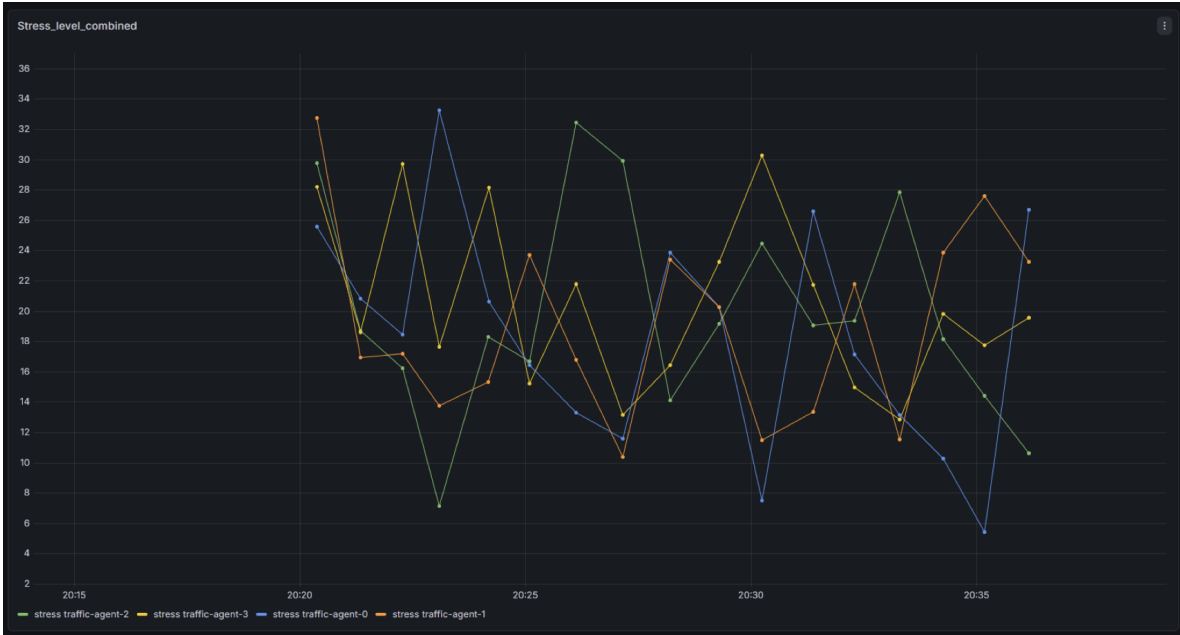


Figure 2: Andamento combinato dello stress index dei Traffic Agent

Dal grafico è possibile osservare oscillazioni differenti tra gli agenti, dovute alla distribuzione dinamica del traffico urbano simulato. Nonostante la presenza di picchi temporanei, il sistema mantiene un comportamento complessivamente stabile.

9.3 Analisi di Scalabilità

Per valutare la scalabilità del sistema sono state eseguite simulazioni aumentando progressivamente il numero di Traffic Agent attivi.

9.3.1 Scenario con 8 agenti



Figure 3: Stress index con 8 Traffic Agent

9.3.2 Scenario con 16 agenti

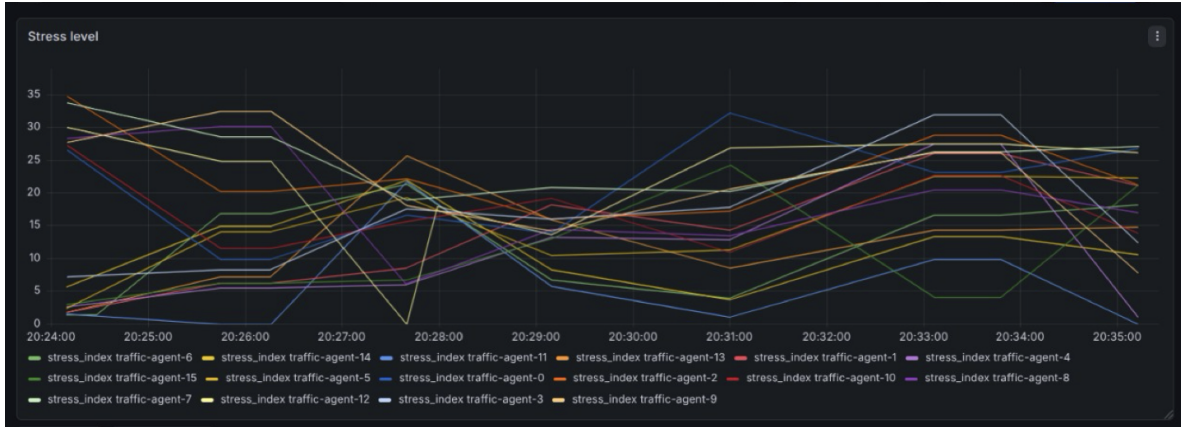


Figure 4: Stress index con 16 Traffic Agent

9.3.3 Scenario con 32 agenti

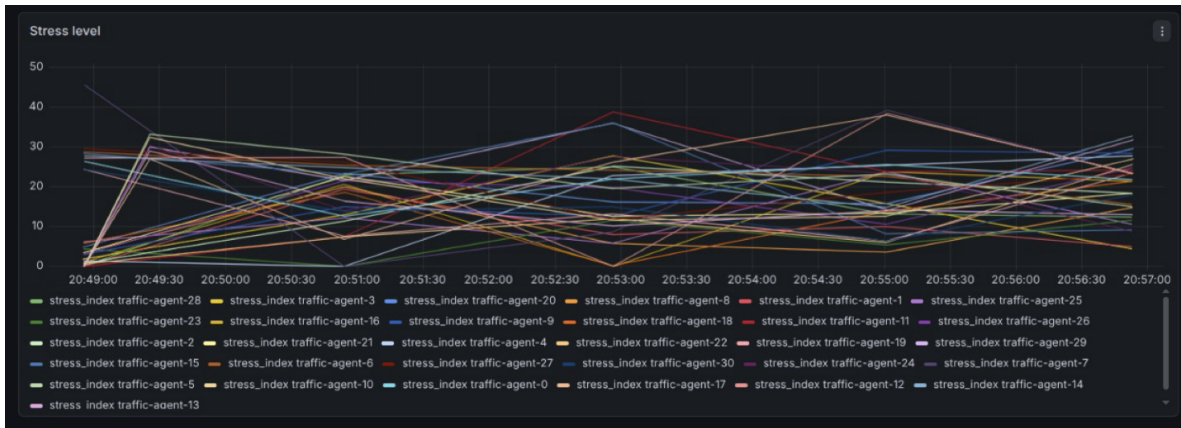


Figure 5: Stress index con 32 Traffic Agent

L'analisi mostra che il sistema mantiene una struttura distribuita stabile anche all'aumentare del numero di agenti, confermando la capacità di Kubernetes di orchestrare correttamente i componenti e di mantenere separazione e isolamento tra i pod.

Nel complesso, i risultati confermano la fattibilità dell'approccio multi-agent distribuito per scenari di controllo adattivo del traffico urbano.

10 Conclusioni

Questo progetto ha mostrato come un'architettura multi-agent supportata da Large Language Models possa essere applicata al controllo intelligente del traffico urbano, mantenendo al tempo stesso proprietà fondamentali di affidabilità, scalabilità e osservabilità.

L'infrastruttura realizzata combina tecnologie cloud-native come Docker, Kubernetes e Minikube con un layer agentico distribuito composto da traffic agent specializzati e da un orchestratore globale. Tale approccio ha permesso di separare il controllo locale del traffico dalla pianificazione strategica globale, favorendo modularità e fault isolation.

L'utilizzo del protocollo MCP ha consentito di esporre capacità operative e memoria condivisa in modo standardizzato e controllato, permettendo agli agenti di accedere allo storico degli stress index e di coordinare le proprie decisioni senza introdurre accoppiamento diretto tra componenti.

Dal punto di vista della reliability engineering, il sistema implementa diversi meccanismi di resilienza:

- isolamento dei componenti tramite pod Kubernetes;
- restart automatico dei pod in caso di crash;
- gestione distribuita degli agenti tramite StatefulSet;
- monitoring centralizzato con Prometheus, Grafana, Loki e Promtail;
- segregazione delle risorse tramite requests/limits;
- controllo dei privilegi tramite RBAC.

L'approccio agentico è stato utilizzato solamente nei punti in cui introduce reale valore operativo, evitando automazioni non controllabili o eccessivamente speculative. Le decisioni critiche rimangono infatti limitate da guardrails espliciti, controllo dei costi inferenziali e politiche di fallback conservative.

Dal punto di vista economico, il sistema presenta un costo operativo relativamente contenuto rispetto ai benefici potenziali ottenibili in termini di riduzione della congestione urbana, miglioramento della mobilità e supporto adattivo al controllo semaforico.

I risultati ottenuti mostrano come l'integrazione tra sistemi distribuiti cloud-native e agenti LLM possa rappresentare una soluzione promettente per scenari real-time complessi, purché supportata da solide politiche di sicurezza, osservabilità e controllo operativo.

Come possibile evoluzione futura, il sistema potrebbe essere esteso introducendo:

- strategie di reinforcement learning per l'ottimizzazione adattiva;
- deployment multi-node Kubernetes;
- simulazioni urbane su larga scala;
- sistemi di human approval dinamico;
- modelli predittivi per eventi di congestione.

In conclusione, il progetto dimostra come gli LLM possano essere utilizzati non come semplici generatori di testo, ma come componenti operativi controllati all'interno di infrastrutture distribuite affidabili e monitorabili.

11 Struttura del Repository

L'intero progetto è organizzato in modo modulare per separare le principali componenti dell'architettura: simulazione SUMO, costruzione delle topologie, backend persistente, agenti locali, orchestratore globale, configurazione Kubernetes e monitoraggio.

Questa struttura riflette il funzionamento distribuito del sistema e consente di mantenere indipendenti i diversi livelli applicativi. In questo modo è possibile modificare o estendere singoli componenti, come agenti, backend, dashboard o manifest Kubernetes, senza compromettere l'intera architettura.

La repository facilita quindi manutenzione, debugging e scalabilità del progetto, rendendo più semplice passare da simulazioni ridotte a deployment multi-agente più estesi.



References

- [1] Kubernetes Documentation, *Production-Grade Container Orchestration*, Available at: <https://kubernetes.io/docs/>
- [2] Docker Documentation, *Docker Container Platform*, Available at: <https://docs.docker.com/>
- [3] Minikube Documentation, *Local Kubernetes Development*, Available at: <https://minikube.sigs.k8s.io/docs/>
- [4] Helm Documentation, *The Package Manager for Kubernetes*, Available at: <https://helm.sh/docs/>
- [5] FastAPI Documentation, *FastAPI Framework*, Available at: <https://fastapi.tiangolo.com/>
- [6] Prometheus Documentation, *Monitoring System and Time Series Database*, Available at: <https://prometheus.io/docs/>
- [7] Grafana Documentation, *Grafana Visualization Platform*, Available at: <https://grafana.com/docs/>
- [8] Grafana Loki Documentation, *Log Aggregation System*, Available at: <https://grafana.com/docs/loki/latest/>
- [9] Promtail Documentation, *Log Collector for Loki*, Available at: <https://grafana.com/docs/loki/latest/send-data/promtail/>
- [10] SQLite Documentation, *SQLite Database Engine*, Available at: <https://www.sqlite.org/docs.html>
- [11] SUMO Documentation, *Simulation of Urban Mobility*, Available at: <https://sumo.dlr.de/docs/>
- [12] TraCI Documentation, *Traffic Control Interface*, Available at: <https://sumo.dlr.de/docs/TraCI.html>
- [13] Model Context Protocol Documentation, *MCP Specification*, Available at: <https://modelcontextprotocol.io/>
- [14] Python Documentation, *Python Programming Language*, Available at: <https://docs.python.org/3/>
- [15] HTTPX Documentation, *Async HTTP Client for Python*, Available at: <https://www.python-httpx.org/>
- [16] Pydantic Documentation, *Data Validation using Python Type Hints*, Available at: <https://docs.pydantic.dev/>
- [17] OpenAI Documentation, *OpenAI API Platform*, Available at: <https://platform.openai.com/docs/>
- [18] Google Gemini Documentation, *Gemini API Documentation*, Available at: <https://ai.google.dev/>
- [19] Bommasani, R. et al., *On the Opportunities and Risks of Foundation Models*, Stanford Center for Research on Foundation Models, 2021.